

Challenge #1: Customers need to add points after being clicked, but can't track it to the ScoreManager script.

In the initial design, I wanted each customer to be able to add points by calling the AddScore() method in the ScoreManager directly after being clicked by the player and served correctly. This way the customer object's own order number could be called directly. And it can be triggered directly when it is clicked, which is more convenient.

However, a structural design problem was quickly encountered. Since the customer is a dynamically generated prefab, it doesn't know whether the ScoreManager exists in the scene or not, and it can't access it directly. Therefore, I decided to change the assignment of responsibilities by moving the "score or not" part of the logic from the customer to the CustomerManager and centralizing it. When the customer is clicked, he/she is only responsible for passing his/her order number and reference to the Manager, which will determine if the current product matches and if the wait time is less than the Quick Serve Threshold. Based on this information, the CustomerManager calls the ScoreManager to add points.

In this way, the customer does not need to know that the score system exists at all. the Manager becomes the center of all the logic, and the system structure is clearer and more reliable.

Challenge #2: The customer needs to be destroyed, but how does the Manager accurately reference this customer object?

Customers need to be destroyed from the scene after an order is completed or after a timeout. I initially considered having the CustomerManager take care of destroying the customer, but this faced a problem: how does the Manager know exactly which customer GameObject to destroy, especially when there are multiple customers at the same time?

To solve this problem, I use GetComponent <Customer>() to get a reference to the customer script when generating the customer, and store this reference in a list of listeners inside the Manager List <Customer>, where each managed customer object corresponds to an explicit instance of Customer. At the same time, when a customer is generated, I have the Manager transfer its reference to the new customer while creating a new customer object. This allows the Customer to in turn notify the Manager to remove itself if it times out, for example.

The Manager can accurately find the customer's reference at any time and listen to it, judge it, or destroy it, and the customer can access the Manager's public methods for status reporting. When clicked, the customer can find the Manager's methods to determine if their order number is the same as the one selected by the player. In the end, you get a direct return of the true or false result.

Challenge #3: Detecting Click Events on Customer Objects

I had previously tried to handle customer click responses through Unity's EventTrigger system. However, I found that EventTrigger cannot be applied to normal game objects without Collider. My customer object is a 2D Sprite in the world space, not a UI, and it can't respond to mouse click events.

To solve this problem, I chose to detect the mouse position when the player clicks the left mouse button and the position of the customer object. If the distance is within a certain range it is judged as a click on that customer. The click request is sent to the CustomerManager for processing.

In the end, this approach successfully implemented the click interaction logic for the customer object.

Challenge #4: How to prevent customers that have been removed from the listener from still being clicked?

Customers will trigger a shrinking animation after a timeout and will be destroyed automatically. In order to prevent the player from clicking on these "gone" customers during the animation, I built an effective click permission control mechanism to make sure

that this is not a problem. I added an `isActive` boolean field for each customer to avoid multiple clicks.

This way, even if the customer hasn't been destroyed yet, as long as he has timed out or been processed, he won't respond to player clicks anymore. Avoiding multiple clicks while the customer is animating.